

The Post LLM → Read to Knowledge

- Know daily mining from LLM:-
 - use prompt to extract general knowledge about entities, actions

• Integrating a reasoner into LLM:-

- use prompts to make the LLM reason step by step (Chain of thoughts)

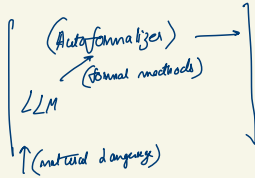
→ These steps are "informal NL" → which is uncontrolled and error prone.

- Build controllable reasoner that uses LLM as tool

Instead of making the LLM do the math, build a strict logical system that calls on the LLM just to fetch new facts or translate text into formulas

1) Avoiding hallucination

2) Converting informal text into "interface-enabling form"



Automated Reasoning

This slide introduces Automated Reasoning as the technology built to mechanically identify valid reasoning → list applications like verifying and solving NP-hard problems. In the context of the paper, this is exactly what Formal Methods are → mathematically rigorous techniques for modelling, reasoning and verifying system.

They are autographic transformers

→ [Model is leaning for Web-based not truth]

→ [Semantic Bridge → not reasoning]

② Architectural Shift

- State fluency
- Deterministic

SMT (Satisfiability Modulo Theory)
Primitive constraints satisfiability

↳ finite set of variables and strict rules governing them.

(Output is strictly binary)

(Ideal for Static Problems) like

(Theorem Proving) → provides unbounded high assurance connections

Difficult to prove, some cases require human intervention.

Model Checking → Behavioral and temporal properties, verify concurrent systems, state machines and protocols over time.

Example:- A Distributed System

• a SMT solver could check if static configuration file don't contradict each other.

• A theorem prover could try to prove if the consensus theorem is mathematically provable

• A model checker does brute force traversal on a state machine directed graph mapping every single permutation of events that the system could ever experience.

It answers that the system never enters a legal state. To do this model checker relies on temporal logic.

(How do you write a request using time)

It gives you the exact sequence of state transition. Exact path in the graph that breaks the system

↓
uses a PAT agent framework [Process Algebra Toolkit]
↳ you need to write (PAT) → becomes hectic.

They came up with Multi Agent LLM - pipeline

There is a self aware engineer who wants to formally model the state machine of our engine.

Step 1 → The engineer literally types a paragraph in English

↓
Step 2 → Planning LLM → uses GPT
Precisely a semantic parser and architect
it uses GPT to deconstruct the simple english to break down the idea of temporal ontology. → (identifies the entities (state variables) and (valid actions) and deduce the two guard condition.

Guard Condition is the strict prerequisite rule that must be there for state transition to occur.

→ Even tho user didn't specify them, the semantic knowledge of LLM becomes massive feature. The next data is being learned on how to deduce the guard condition

The drive fails, it can't check for the fact that key can go out even if the driver is inside.

[Engineer don't have to step into] Solve this, the France work is fully automated.

② Repair agent LLM

↳ checks for all issues and then inject the patches

↓
This iteration continues until all issues are fixed.

How does refinement calculus deal with this in traditional development you write operational code. but in refinement calculus a program is viewed mathematically as predicate frames former.

→ (Pre condition) → (Post condition)

Refinement how → (Abstract → concrete)
(every single time you apply refinement calculus) → (not great) →

Don't do the formal specification of compiler, which is done in Assembly type not

Main - if even would be if the LLM initially interpreted the idea wrong. Because of even if formal methods and verifications are correct. they might be in correct.

How does Paper detect/deals with

non NIDE

↳ N-version (auto formalization)

↳ Multiple instances write the same thing
(temperature variance)

↳ if agree → ✓

↳ if disagree →

↳ routes + specialist needs a task prompt to domain check.

9) Why this study in the first place?

→ The LLM's are extremely capable of natural language understanding, coding, planning, reasoning. But their output remains probabilistic. Fundamental limitation of the probabilistic nature of LLM.

Formal methods: - have opposite characteristics they are mathematically rigorous deterministic

So can we use LLM for flexibility and generation while using formal methods as correctness

8) About today's clouds, chatgpt and all already relevant and up to date? How do they work? Do they also rely on Symbolic AI type reasoning in the backend.

Modern models such as chatgpt, clouds and gemini are already better at reasoning why this study? (Better reasoning) vs (Generated content)

A larger model may dramatically increase the probability of producing the correct answer.

Do current models like clouds, gemini, chatgpt use formal methods internally?

Current LLM → use tool such as code execution, calculation, web search, etc. Tools substantially improve reliability.

Example LLM generate code → execute it → see result - (tool) Fix it

But it does not prove if program is correct or not

large Model and Small model

Can a small model when combined with formal verification achieve comparable reliability to a larger model?

Bench Mark, accuracy, time, tokens, memory, parameters

c) (Code generation) → Can we divide the idea of code into different category and tackle them?

Code according to importance:-

- low risk code (UI-components)
- high risk code (Business logic)
- Safety-critical code (Security code)

different types of code have fundamentally different verification.

Code Classification by verification Process

Artifact	Important Property	Possible Verification
UI code	Basic functional correctness	Test / Static checks.
Business logic	Constraints / Invariants	SMT / contracts / test
Security component	Security Properties	Static analysis / formal verification.

(Adaptive Verification Systems)

When we should integrate formal methods and why?

→ How of a general architecture - selection problem

Before trying hard why accuracy is important? We need to know when/where/which tasks (hallucination could be disruptive) and LLM are directly / indirectly involved.

→ Directly / Indirectly I mean → LLM as interface:- LLM mostly translate natural language for a deterministic backend.

LLM as core executor:- how LLM actually produces the reasoning that determines the output.

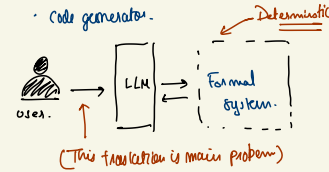
LLM → as an interface

Examples:-

Autonomous vehicles, other safety critical system
Symbolic / formal systems serves the main backend.

But LLM: could be a interface. like

- Natural-language interface
- Planner.
- Specification generator.
- Code generator.



(LLM as reasoning)

- Code generation
- Theorem proving
- Legal reasoning
- Medical analysis

→ Deterministic Based

→ (RAG Based)

Ques: Can we verify that retrieval is 100% correct?

The problem is that the relevance itself isn't naturally a binary mathematical property

- more of a semantic judgement
- so instead of prove that retrieval is correct we need to ask can we define and verify necessary properties that retrieval must satisfy.

Satisfy. [RAI Based Reasoning, Medical, Legal, etc.]

→ Knowledge graph is not the central idea

→ Can LLM correctly understand a complex problem and map it to correct semantic concept in the knowledge base.

Hallucination is Not a problem

Hallucination can mean different things

① Knowledge hallucination:-

the model invents a fact

② Retrieval failure:-

the correct evidence exist but isn't retrieved.

③ Semantic interpretation failure:-

The model mis understands the con.

④ Reasoning failure:-

the model has the right facts but reasons incorrectly

⑤ Formalization failure:-

The model converts the intended meaning into the wrong formal representation

⑥ Generation failure:-

the formal reasoning is correct but the generated artifact is wrong

⑦ Specification failure:-

The formal specification itself doesn't capture the user's actual intention

⑧ Most important failure:-

correctly verifying the wrong thing.

② When hallucination is the problem: (examples) :-

① Fact based hallucination :- (data, News, information, insurance would be big deal)
Maybe a proper rag system can deal with it.

single
LLM →

② Solve Problems :-

- ① Physics,
- ② Maths,
- ③ (Reasoning Problem)

} Static / Dynamic

③ Write codes (Syntax match + correct intention)

④ Do task

⑤ → Break a problem to small task and deal with it.